JavaScript is disabled on your browser.



compact1, compact2, compact3

java.util.concurrent

# Class LinkedTransferQueue<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>
      java.util.concurrent.LinkedTransferQueue<E>

**Type Parameters:**

`E - the type of elements held in this collection`

**All Implemented Interfaces:**

`Serializable, Iterable<E>, Collection<E>, BlockingQueue<E>, TransferQueue<E>, Queue<E>`

---

```
public class LinkedTransferQueue<E>
extends AbstractQueue<E>
implements TransferQueue<E>, Serializable
```

An unbounded `TransferQueue` based on linked nodes. This queue orders elements FIFO (first-in-first-out) with respect to any given producer. The *head* of the queue is that element that has been on the queue the longest time for some producer. The *tail* of the queue is that element that has been on the queue the shortest time for some producer.

Beware that, unlike in most collections, the `size` method is *NOT* a constant-time operation. Because of the asynchronous nature of these queues, determining the current number of elements requires a traversal of the elements, and so may report inaccurate results if this collection is modified during traversal. Additionally, the bulk operations `addAll`, `removeAll`, `retainAll`, `containsAll`, `equals`, and `toArray` are *not* guaranteed to be performed atomically. For example, an iterator operating concurrently with an `addAll` operation might view only some of the added elements.

This class and its iterator implement all of the *optional* methods of the `Collection` and `Iterator` interfaces.

Memory consistency effects: As with other concurrent collections, actions in a thread prior to placing an object into a `LinkedTransferQueue` *happen-before* actions subsequent to the access or removal of that element from the `LinkedTransferQueue` in another thread.

This class is a member of the Java Collections Framework.

**Since:**

`1.7`

**See Also:**

`Serialized Form`

## *Constructor Summary*

**Constructors**

| Constructor | Description |
|---|---|
| `LinkedTransferQueue()` | Creates an initially empty `LinkedTransferQueue`. |
| `LinkedTransferQueue(Collection<? extends E> c)` | Creates a `LinkedTransferQueue` initially containing the elements of the given collection, added in traversal order of the collection's iterator. |

## *Method Summary*

**Concrete Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| `boolean` | `add(E e)` | Inserts the specified element at the tail of this queue. |
| `boolean` | `contains(Object o)` | Returns `true` if this queue contains the specified element. |
| `int` | `drainTo(Collection<? super E> c)` | Removes all available elements from this |

| | | |
|---|---|---|
| | | queue and adds them to the given collection. |
| int | **drainTo**(**Collection**<? super **E**> c, int maxElements) | Removes at most the given number of available elements from this queue and adds them to the given collection. |
| int | **getWaitingConsumerCount**() | Returns an estimate of the number of consumers waiting to receive elements via **BlockingQueue.take()** or timed **poll**. |
| boolean | **hasWaitingConsumer**() | Returns `true` if there is at least one consumer waiting to receive an element via **BlockingQueue.take()** or timed **poll**. |
| boolean | **isEmpty**() | Returns `true` if this queue contains no elements. |
| **Iterator**<**E**> | **iterator**() | Returns an iterator over the elements in this queue in proper sequence. |
| boolean | **offer**(**E** e) | Inserts the specified element at the tail of this queue. |
| boolean | **offer**(**E** e, long timeout, **TimeUnit** unit) | Inserts the specified element at the tail of this queue. |
| **E** | **peek**() | Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty. |

| | | |
|---|---|---|
| **E** | **poll**() | Retrieves and removes the head of this queue, or returns `null` if this queue is empty. |
| **E** | **poll**(long timeout, **TimeUnit** unit) | Retrieves and removes the head of this queue, waiting up to the specified wait time if necessary for an element to become available. |
| void | **put**(**E** e) | Inserts the specified element at the tail of this queue. |
| int | **remainingCapacity**() | Always returns `Integer.MAX_VALUE` because a `LinkedTransferQueue` is not capacity constrained. |
| boolean | **remove**(**Object** o) | Removes a single instance of the specified element from this queue, if it is present. |
| int | **size**() | Returns the number of elements in this queue. |
| **Spliterator**<**E**> | **spliterator**() | Returns a **Spliterator** over the elements in this queue. |
| **E** | **take**() | Retrieves and removes the head of this queue, waiting if necessary until an element becomes available. |
| void | **transfer**(**E** e) | Transfers the element to a consumer, waiting if necessary to do so. |
| boolean | **tryTransfer**(**E** e) | Transfers the element to a waiting consumer immediately, if possible. |
| boolean | | |

| | | |
|---|---|---|
| | **tryTransfer**(**E** e, long timeout, **TimeUnit** unit) | Transfers the element to a consumer if it is possible to do so before the timeout elapses. |

### Methods inherited from class java.util.AbstractQueue

addAll, clear, element, remove

### Methods inherited from class java.util.AbstractCollection

containsAll, removeAll, retainAll, toArray, toArray, toString

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

### Methods inherited from interface java.util.Queue

element, remove

### Methods inherited from interface java.util.Collection

addAll, clear, containsAll, equals, hashCode, parallelStream, removeAll, removeIf, retainAll, stream, toArray, toArray

### Methods inherited from interface java.lang.Iterable

forEach

## *Constructor Detail*

### LinkedTransferQueue

```
public LinkedTransferQueue()
```

Creates an initially empty `LinkedTransferQueue`.

### LinkedTransferQueue

```
public LinkedTransferQueue(Collection<? extends E> c)
```

Creates a LinkedTransferQueue initially containing the elements of the given collection, added in traversal order of the collection's iterator.

**Parameters:**

c - the collection of elements to initially contain

**Throws:**

NullPointerException - if the specified collection or any of its elements are null

## *Method Detail*

### spliterator

public Spliterator<E> spliterator()

Returns a Spliterator over the elements in this queue.

The returned spliterator is *weakly consistent*.

The Spliterator reports Spliterator.CONCURRENT, Spliterator.ORDERED, and Spliterator.NONNULL.

**Specified by:**

spliterator in interface Iterable<E>

**Specified by:**

spliterator in interface Collection<E>

**Implementation Note:**

The Spliterator implements trySplit to permit limited parallelism.

**Returns:**

a Spliterator over the elements in this queue

**Since:**

1.8

### put

public void put(E e)

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never block.

**Specified by:**

put in interface BlockingQueue<E>

**Parameters:**

e - the element to add

**Throws:**

NullPointerException - if the specified element is null

### offer

```
public boolean offer(E e,
                     long timeout,
                     TimeUnit unit)
```

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never block or return `false`.

**Specified by:**

offer in interface BlockingQueue<E>

**Parameters:**

e - the element to add

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

**Returns:**

true (as specified by BlockingQueue.offer)

**Throws:**

NullPointerException - if the specified element is null

### offer

```
public boolean offer(E e)
```

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never return `false`.

**Specified by:**

offer in interface BlockingQueue<E>

**Specified by:**

offer in interface Queue<E>

**Parameters:**

e - the element to add

**Returns:**

true (as specified by Queue.offer(E))

**Throws:**

NullPointerException - if the specified element is null

### add

public boolean add(E e)

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never throw IllegalStateException or return false.

**Specified by:**

add in interface Collection<E>

**Specified by:**

add in interface BlockingQueue<E>

**Specified by:**

add in interface Queue<E>

**Overrides:**

add in class AbstractQueue<E>

**Parameters:**

e - the element to add

**Returns:**

true (as specified by Collection.add(E))

**Throws:**

NullPointerException - if the specified element is null

### tryTransfer

public boolean tryTransfer(E e)

Transfers the element to a waiting consumer immediately, if possible.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in take() or timed poll), otherwise returning false without enqueuing the element.

**Specified by:**

tryTransfer in interface TransferQueue<E>

**Parameters:**

e - the element to transfer

**Returns:**

true if the element was transferred, else false

**Throws:**

NullPointerException - if the specified element is null

### transfer

```
public void transfer(E e)
              throws InterruptedException
```

Transfers the element to a consumer, waiting if necessary to do so.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `take()` or timed `poll`), else inserts the specified element at the tail of this queue and waits until the element is received by a consumer.

**Specified by:**

`transfer` in interface `TransferQueue<E>`

**Parameters:**

`e` - the element to transfer

**Throws:**

`NullPointerException` - if the specified element is null

`InterruptedException` - if interrupted while waiting, in which case the element is not left enqueued

### tryTransfer

```
public boolean tryTransfer(E e,
                           long timeout,
                           TimeUnit unit)
                    throws InterruptedException
```

Transfers the element to a consumer if it is possible to do so before the timeout elapses.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `take()` or timed `poll`), else inserts the specified element at the tail of this queue and waits until the element is received by a consumer, returning `false` if the specified wait time elapses before the element can be transferred.

**Specified by:**

`tryTransfer` in interface `TransferQueue<E>`

**Parameters:**

`e` - the element to transfer

`timeout` - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

**Returns:**

true if successful, or false if the specified waiting time elapses before completion, in which case the element is not left enqueued

**Throws:**

NullPointerException - if the specified element is null

InterruptedException - if interrupted while waiting, in which case the element is not left enqueued

### take

```
public E take()
       throws InterruptedException
```

**Description copied from interface: BlockingQueue**

Retrieves and removes the head of this queue, waiting if necessary until an element becomes available.

**Specified by:**

take in interface BlockingQueue<E>

**Returns:**

the head of this queue

**Throws:**

InterruptedException - if interrupted while waiting

### poll

```
public E poll(long timeout,
              TimeUnit unit)
       throws InterruptedException
```

**Description copied from interface: BlockingQueue**

Retrieves and removes the head of this queue, waiting up to the specified wait time if necessary for an element to become available.

**Specified by:**

poll in interface BlockingQueue<E>

**Parameters:**

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

**Returns:**

the head of this queue, or null if the specified waiting time elapses before an element is available

**Throws:**

`InterruptedException` - if interrupted while waiting

### poll

```
public E poll()
```

**Description copied from interface: Queue**

Retrieves and removes the head of this queue, or returns `null` if this queue is empty.

**Specified by:**

`poll` in interface `Queue<E>`

**Returns:**

the head of this queue, or null if this queue is empty

### drainTo

```
public int drainTo(Collection<? super E> c)
```

**Description copied from interface: BlockingQueue**

Removes all available elements from this queue and adds them to the given collection. This operation may be more efficient than repeatedly polling this queue. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

**Specified by:**

`drainTo` in interface `BlockingQueue<E>`

**Parameters:**

`c` - the collection to transfer elements into

**Returns:**

the number of elements transferred

**Throws:**

`NullPointerException` - if the specified collection is null

`IllegalArgumentException` - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

### drainTo

```
public int drainTo(Collection<? super E> c,
                   int maxElements)
```

**Description copied from interface: BlockingQueue**

Removes at most the given number of available elements from this queue and adds them to the given collection. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

**Specified by:**

`drainTo` in interface `BlockingQueue<E>`

**Parameters:**

`c` - the collection to transfer elements into

maxElements - the maximum number of elements to transfer

**Returns:**

the number of elements transferred

**Throws:**

NullPointerException - if the specified collection is null

IllegalArgumentException - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

### iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this queue in proper sequence. The elements will be returned in order from first (head) to last (tail).

The returned iterator is *weakly consistent*.

**Specified by:**

iterator in interface Iterable<E>

**Specified by:**

iterator in interface Collection<E>

**Specified by:**

iterator in class AbstractCollection<E>

**Returns:**

an iterator over the elements in this queue in proper sequence

### peek

```
public E peek()
```

**Description copied from interface: Queue**

Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.

**Specified by:**

peek in interface Queue<E>

**Returns:**

the head of this queue, or null if this queue is empty

### isEmpty

```
public boolean isEmpty()
```

Returns `true` if this queue contains no elements.

**Specified by:**

`isEmpty` in interface `Collection<E>`

**Overrides:**

`isEmpty` in class `AbstractCollection<E>`

**Returns:**

`true if this queue contains no elements`

### hasWaitingConsumer

```
public boolean hasWaitingConsumer()
```

**Description copied from interface: `TransferQueue`**

Returns `true` if there is at least one consumer waiting to receive an element via `BlockingQueue.take()` or timed `poll`. The return value represents a momentary state of affairs.

**Specified by:**

`hasWaitingConsumer` in interface `TransferQueue<E>`

**Returns:**

`true if there is at least one waiting consumer`

### size

```
public int size()
```

Returns the number of elements in this queue. If this queue contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Beware that, unlike in most collections, this method is *NOT* a constant-time operation. Because of the asynchronous nature of these queues, determining the current number of elements requires an O(n) traversal.

**Specified by:**

`size` in interface `Collection<E>`

**Specified by:**

`size` in class `AbstractCollection<E>`

**Returns:**

the number of elements in this queue

### getWaitingConsumerCount

```
public int getWaitingConsumerCount()
```

**Description copied from interface: `TransferQueue`**

Returns an estimate of the number of consumers waiting to receive elements via `BlockingQueue.take()` or timed `poll`. The return value is an approximation of a momentary state of affairs, that may be inaccurate if consumers have completed or given up waiting. The value may be useful for monitoring and heuristics, but not for synchronization control. Implementations of this method are likely to be noticeably slower than those for `TransferQueue.hasWaitingConsumer()`.

**Specified by:**

`getWaitingConsumerCount` in interface `TransferQueue<E>`

**Returns:**

the number of consumers waiting to receive elements

### remove

```
public boolean remove(Object o)
```

Removes a single instance of the specified element from this queue, if it is present. More formally, removes an element `e` such that `o.equals(e)`, if this queue contains one or more such elements. Returns `true` if this queue contained the specified element (or equivalently, if this queue changed as a result of the call).

**Specified by:**

`remove` in interface `Collection<E>`

**Specified by:**

`remove` in interface `BlockingQueue<E>`

**Overrides:**

`remove` in class `AbstractCollection<E>`

**Parameters:**

`o` - element to be removed from this queue, if present

**Returns:**

`true` if this queue changed as a result of the call

### contains

`public boolean contains(Object o)`

Returns `true` if this queue contains the specified element. More formally, returns `true` if and only if this queue contains at least one element `e` such that `o.equals(e)`.

**Specified by:**

`contains` in interface `Collection<E>`

**Specified by:**

`contains` in interface `BlockingQueue<E>`

**Overrides:**

`contains` in class `AbstractCollection<E>`

**Parameters:**

`o` - object to be checked for containment in this queue

**Returns:**

`true` if this queue contains the specified element

### remainingCapacity

```
public int remainingCapacity()
```

Always returns `Integer.MAX_VALUE` because a `LinkedTransferQueue` is not capacity constrained.

**Specified by:**

`remainingCapacity` in interface `BlockingQueue<E>`

**Returns:**

`Integer.MAX_VALUE` (as specified by `BlockingQueue.remainingCapacity`)




Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.